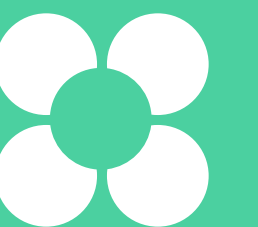
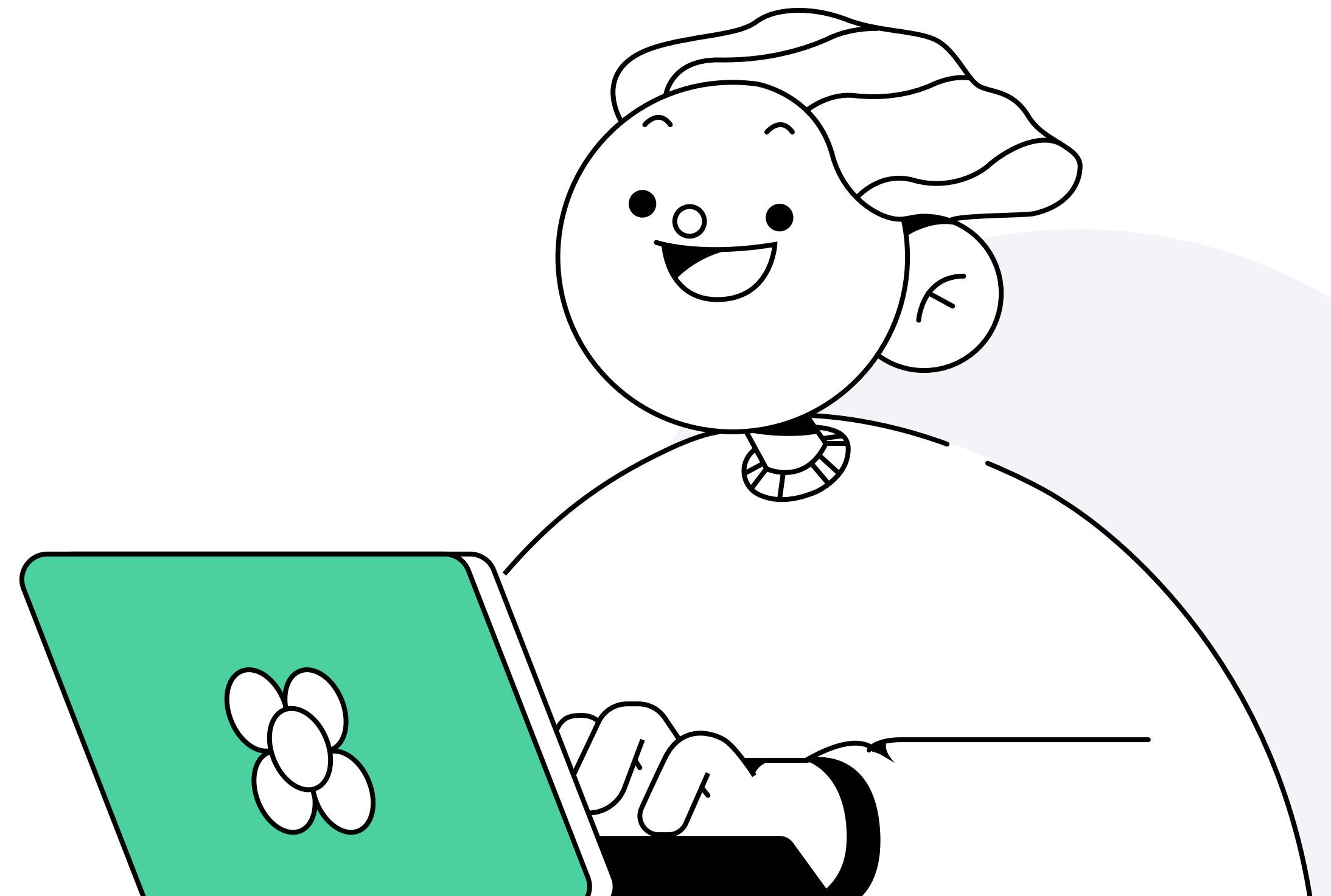


# Многомерные массивы

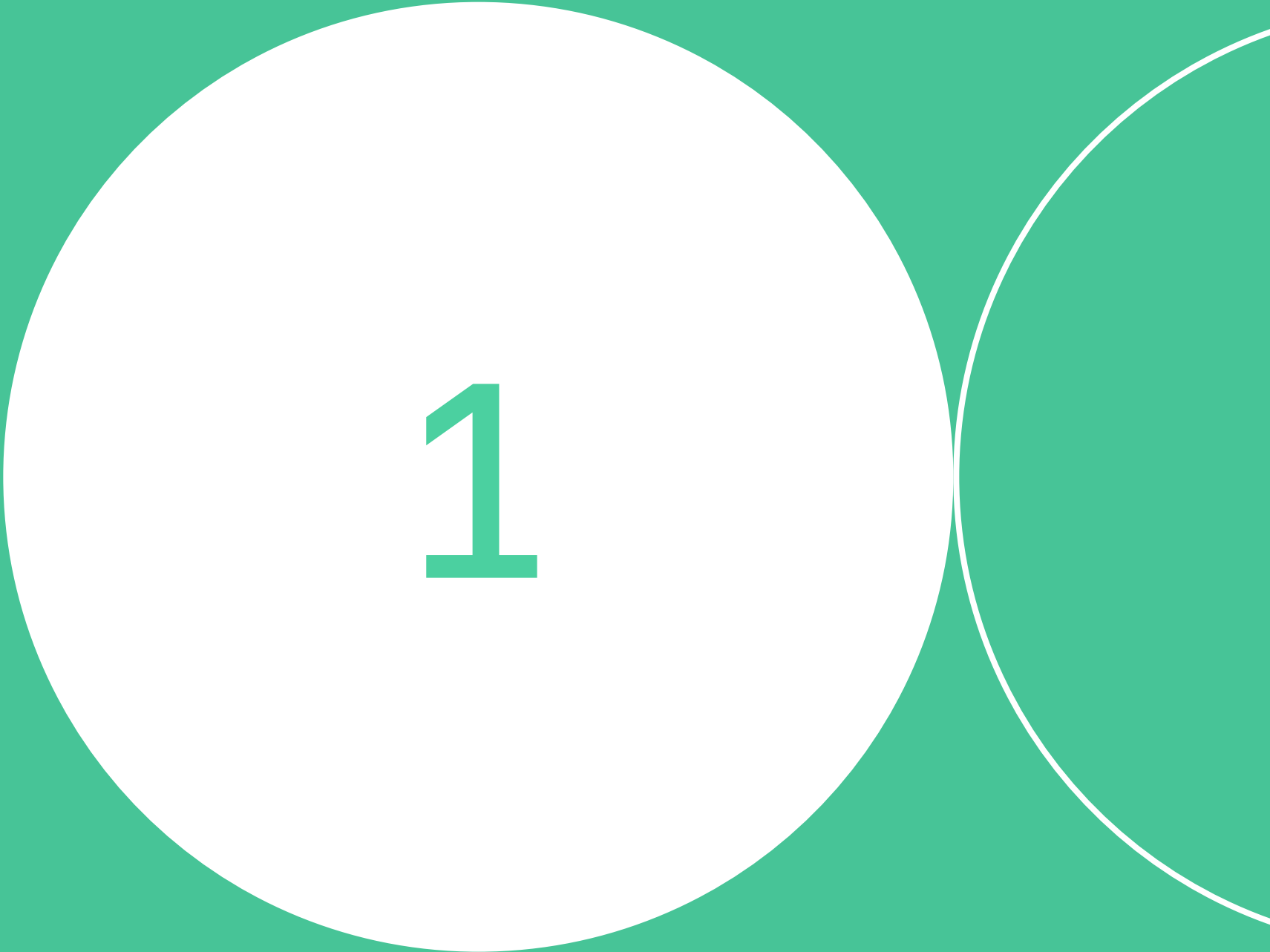


# План занятия

- ① Структура двумерного массива
- ② Инициализация двумерного массива



# Структура двумерного массива

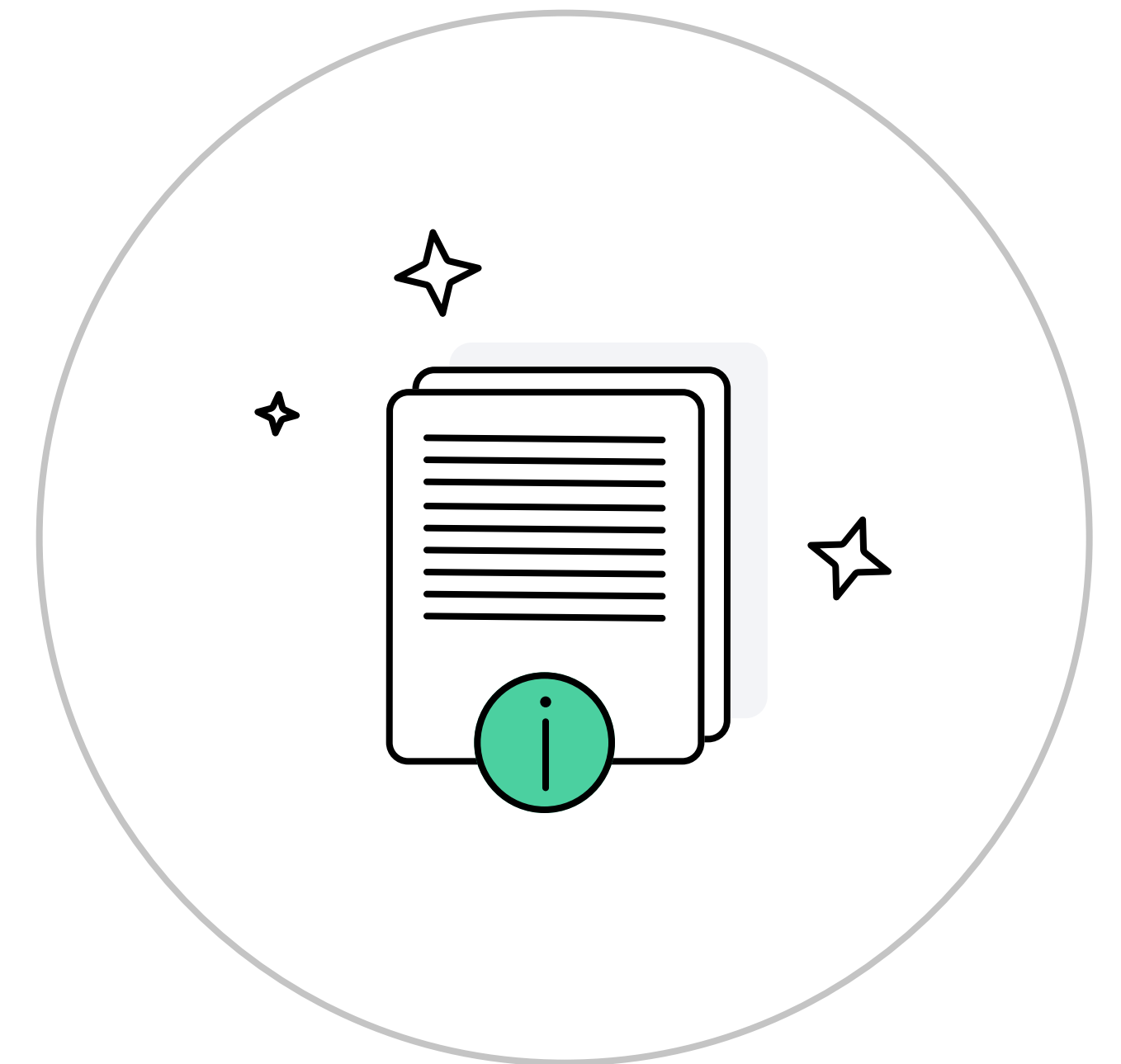


1

# Структура двумерного массива

Многомерные массивы чаще всего используются для проектирования пространства в различных двумерных игрушках.

Массив считается **двумерным**, если для определения местоположения элемента в массиве нужно указать значения двух индексов



# Структура двумерного массива

Двумерный массив ассоциируется с таблицей. Если количество столбцов и строк массива (таблицы) одинаковое, то массив ещё называют **матрицей**.

Массив считается **многомерным**, если для определения местоположения элемента в массиве нужно указать значения трёх и более индексов.

- Далее мы рассмотрим примеры для двумерного массива, который встречается чаще всего. С многомерными массивами можно работать так же, как и с двумерными, но только учитывая количество индексов



# Структура двумерного массива

При объявлении многомерного массива указываем тип, размеры и имя. Массив можно создать тремя способами:

```
тип имя_переменной[][] = new тип[размер][размер];  
тип[][] имя_переменной = new тип[размер][размер];  
тип[] имя_переменной[] = new тип[размер][размер];
```

- **тип** — базовый тип элементов массива
- **размер** — число элементов массива (выделение памяти для элементов)
- ✳ Элементы массива могут быть любого типа, допустимого в языке Java

# Структура двумерного массива

После определения размера выделяется память для массива. Выделенная память не может быть пустой — для разных типов данных память заполняется разными значениями.

При выделении памяти элементы массива заполняются:

- для числовых типов — нулевыми значениями
- для логического типа **boolean** — значениями **false**
- для ссылочных типов — значениями **null**

# Инициализация двумерного массива



2

# Инициализация двумерного массива

Создадим двумерный массив типа `int` и выведем его значение на экран с помощью метода **`deepToString`**

```
int[][] arr2 = new int [3][];  
arr2[0] = new int [1];  
arr2[1] = new int [4];  
arr2[2] = new int [2];  
  
System.out.println(Arrays.deepToString(arr2));  
// в результате получим [[0], [0, 0, 0, 0], [0, 0]]
```

# Инициализация двумерного массива

Из примера мы видим, что у нас в ячейках лежат три одномерных массива разного размера:

```
[[0], [0, 0, 0, 0], [0, 0]]
```

# Инициализация двумерного массива

Чтобы обратиться к внутренним ячейкам, в которых лежат уже непосредственно числа, нам необходимо выполнить двойную индексацию. Пропишем дополнительную команду:

```
int[][] arr2 = new int [3][];  
arr2[0] = new int [1];  
arr2[1] = new int [4];  
arr2[2] = new int [2];  
  
arr2[1][3] = 15;  
  
System.out.println(Arrays.deepToString(arr2));  
// в результате получим [[0], [0, 0, 0, 15], [0, 0]]
```

# Инициализация двумерного массива

Если при создании массива в правой части кода мы заполним вторые квадратные скобки, то Java создаст внутренние массивы для каждой ячейки с размером, указанным в этих скобках:

```
int[][] arr2 = new int [3][5];
```

```
arr2[1][3] = 15;
```

```
System.out.println(Arrays.deepToString(arr2));
```

```
// в результате получим [[0, 0, 0, 0, 0], [0, 0, 0, 15, 0], [0, 0, 0, 0, 0]]
```

# Инициализация двумерного массива

Рассмотрим ещё один способ создания массива с помощью фигурных скобок:

```
int[][] arr2 = {  
    { 1 },  
    { 22, 222 },  
    { 333, 345, 478 }  
};
```

```
System.out.println(Arrays.deepToString(arr2));
```

```
// в результате получим [[1], [22, 222], [333, 345, 478]]
```

# Инициализация двумерного массива

Выведем данные двумерного массива в виде таблицы с помощью цикла for.

```
// Определяем значения массива
// Цикл по первой размерности выводит строки
for (int i = 0; i < arr2.length; i++) {
    // Цикл по второй размерности выводит колонки — вывод одной строки
    for (int j = 0; j < arr2[i].length; j++) {
        // Используем оператор print — без перехода на следующую строку
        System.out.print(arr2[i][j] + " ");
    }
    // Переход на следующую строку
    System.out.println();
}
```

# Инициализация двумерного массива

В результате мы получим:

```
1  
22, 222  
333, 345, 478
```

# Многомерный массив

Объявление многомерного массива представляет из себя описание массива с таким количеством [ ], которое необходимо для описания количества ключей, по которым может быть найден элемент.

Например, описание трёхмерного массива выглядит так:

```
String products[][][] = new String[2][4][5];
```

⊛ Многомерные массивы используются крайне редко